الجامعة المصرية اليابانية للعلوم و التكنولوجيا
エジプト日本科学技術大学
EGYPT-JAPAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

الجامعة المصرية اليابانية للعلوم و التكنولوجيا
エジプト日本科学技術大学
EGYPT-JAPAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Egypt Japan University Of Science & Technology

Faculty of Engineering

Course project : Omni-Directional Mobile Platform and Ball Balancing System

Mohamed Abdelsabour 120220013
Kareem Shaban Eid 120230257
Mohamed Gamal Zain 120220089
Mohammed Hamada 120220265

Program: Mechatronics Engineering
Sensors and Actuators (MTE 434)
fall - 2025

Submitted to:
Prof. Mohamed Adel
Eng. Gana Elemam

Scientific Report: Integrated Omni-Directional Mobile Platform and Ball Balancing System

1. Introduction

The integration of complex control systems into a unified robotic platform presents a significant challenge in mechatronics, demanding a deep understanding of kinematics, non-linear control, and sensor-actuator interfacing. This report details the design, implementation, and control of a novel two-part robotic system: a **three-wheeled omni-directional mobile base** and an integrated **ball balancing system**. The platform serves as a robust testbed for demonstrating advanced concepts in mobile robotics and real-time stabilization, with a particular focus on the principles of **Sensors and Actuators**.

The mobile base provides **holonomic movement**, allowing for instantaneous translation and rotation, which is controlled via a layered architecture: ROS2 teleoperation, a Raspberry Pi central processor, an Arduino microcontroller for low-level control, and L298N motor drivers. The ball balancing system, conversely, addresses an inherently unstable, non-linear control problem by employing a **resistive touch panel** as the primary sensor and **RC servo motors** as the actuators to maintain the ball's equilibrium.

This report is structured to provide the in-depth technical analysis required for a Sensors and Actuators course project. The full source code for the control systems is provided in the **Technical Appendix** for comprehensive review.

2. Literature Review

2.1. Omni-Directional Mobile Robotics and Control Stack

The three-wheeled omni-directional configuration is a popular choice for mobile platforms due to its simplicity and full mobility in the 2D plane [1]. The control of such a system is typically hierarchical. At the highest level, the **Robot Operating System 2 (ROS2)** provides a standardized framework for communication and teleoperation, allowing a user to send velocity commands (e.g., via teleop_twist_keyboard) [2]. The **Raspberry Pi** acts as the host for ROS2, interpreting these high-level commands.

The translation from desired velocity to motor command requires the application of **inverse kinematics** (detailed in Section 6.1). This calculation is then passed to a low-level microcontroller, the **Arduino**, via serial communication. The Arduino's role is critical for the time-sensitive task of generating the Pulse Width Modulation (PWM) signals required by the **L298N H-bridge motor driver** to control the three DC motors. This architecture effectively separates the complex computational tasks (ROS2, Kinematics) from the real-time actuation tasks (PWM generation).

2.2. Ball Balancing Systems and Sensing Modalities

The ball-on-plate system is a classical non-linear control problem often used to benchmark control algorithms [3]. The system's instability necessitates a highly responsive sensor and actuator system. Historically, two main sensing modalities have been employed:

1. **Camera-Based Vision Systems**: These systems use an overhead camera and image processing (e.g., Hough transforms) to detect the ball's position [4]. While offering high spatial resolution, they are computationally intensive, susceptible to lighting variations, and introduce significant latency due to image acquisition and processing time.
2. **Resistive Touch Panels**: As demonstrated in several studies, the resistive touch panel offers a cost-effective and low-latency alternative [5] [6]. By directly measuring the electrical resistance change caused by the ball's contact, the system bypasses the need for complex computer vision, making it ideal for real-time, microcontroller-based control loops.

The control strategy for the ball balancing system almost universally relies on a **PID controller** due to its proven effectiveness in stabilizing unstable systems.

3. System Architecture

The integrated robot system is a modular design, allowing for independent control of the mobile base and the ball balancing platform. The two subsystems are physically coupled but controlled by a unified electronic core.

Table 1: Integrated System Component Summary

Subsystem	Function	Key Components	Sensor/Actuator Focus	Control Method
Mobile Base	Holonomic movement and teleoperation	3 Omni-wheels, DC Motors, L298N, Arduino, Raspberry Pi	L298N (Actuator), DC Motors (Actuator)	Inverse Kinematics, ROS2 Teleop
Ball Balancing	Real-time stabilization of a ball	Sensitive Plate (Touch Panel), Servos, Ball	Resistive Touch Panel (Sensor), RC Servos (Actuator)	PID Control

The **Arduino** serves as the low-level hardware abstraction layer, receiving high-level commands from the **Raspberry Pi** (for the base) and executing the real-time control loop (for the ball balancing system).

Photo of the complete integrated robot system (Mobile Base + Ball Balancing Platform)

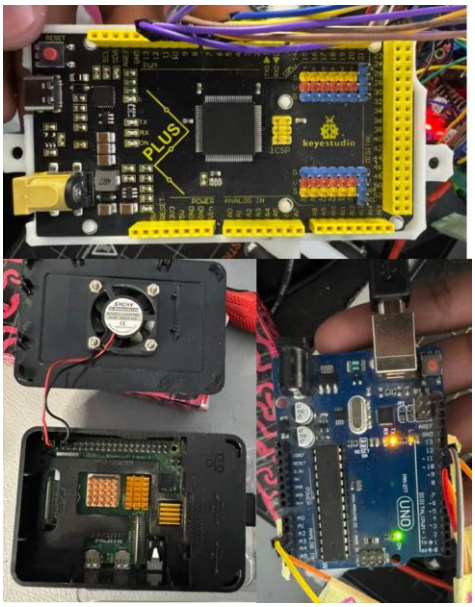

Photo of the Raspberry Pi and Arduino microcontrollers used in the system

4. Sensor Deep Dive: The Resistive Touch Panel

The sensitive plate used for ball position detection is a **4-wire resistive touch panel**. This sensor is crucial for the ball balancing system, as its performance directly dictates the stability and responsiveness of the entire control loop.

4.1. Physical Construction and Principle of Operation

A 4-wire resistive touch panel consists of two electrically conductive layers separated by microscopic insulating spacers [11].

3 **Top Layer (Film)**: A flexible film (typically PET) coated with a transparent conductive material, **Indium Tin Oxide (ITO)**, on its inner surface.
4 **Bottom Layer (Substrate)**: A rigid substrate (glass or acrylic) also coated with ITO.

The principle of operation is based on the **voltage divider rule** and the measurement of resistance. When the ball makes contact with the plate, the flexible top layer is pressed down, causing the ITO coatings on the two layers to meet at the point of contact. This contact point effectively closes a circuit, allowing the system to measure the voltage at that specific location.

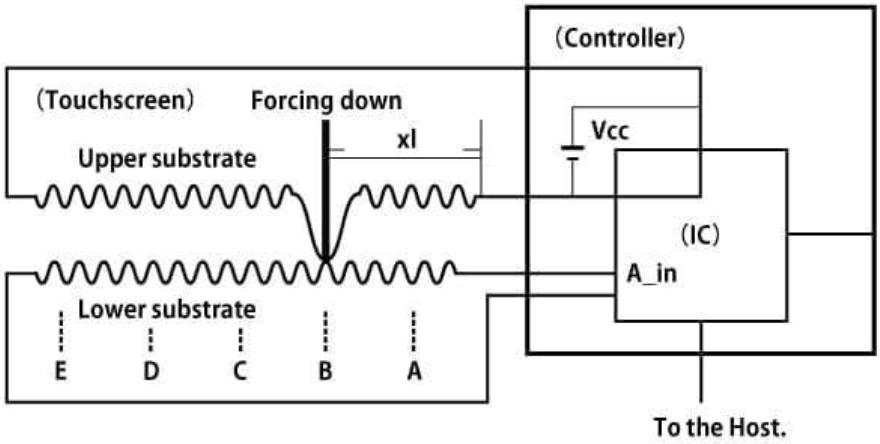

Diagram illustrating the 4-wire resistive touch panel principle

4.2. Coordinate Measurement Technique

The 4-wire configuration uses a sequential, time-multiplexed method to determine the X and Y coordinates of the contact point. This process requires the microcontroller (Arduino) to dynamically reconfigure its I/O pins between output (voltage drive) and input (voltage read).

In the provided implementation, the TouchScreen library is utilized to abstract the low-level pin switching and ADC reading. The ts.getPoint() function performs the sequence of driving one layer and reading from the other to return a TSPoint object containing the raw ADC values for X, Y, and the pressure (Z).

```
// Coordinate Mapping Snippet (C++)
```

```
p = ts.getPoint();
if (p.z >= 10) {
  float x = map(p.x, 0, 1024, -82, 82);
  float y = map(p.y, 0, 1024, 75.5, -75.5);
  // ... filtering and PID logic ...
}
```

The raw ADC values (0-1024) are mapped to physical dimensions (in millimeters) to provide a consistent coordinate system for the PID controller.

4.3. Signal Filtering: Moving Average Filter

To mitigate the inherent noise in resistive touch sensors, a **Moving Average Filter** is implemented. This filter smooths the positional data by averaging the last 10 readings, effectively reducing high-frequency jitter that could destabilize the derivative term of the PID controller.

5. Actuators Deep Dive: RC Servo Motors and L298N Driver

The system employs two distinct types of actuators: **RC Servo Motors** for the ball balancing platform and **DC Motors** controlled by an **L298N H-bridge** for the mobile base.

5.1. RC Servo Motors (Ball Balancing Actuation)

The RC servo motors are the primary actuators for the ball balancing system, responsible for precisely controlling the plate's pitch and roll angles.

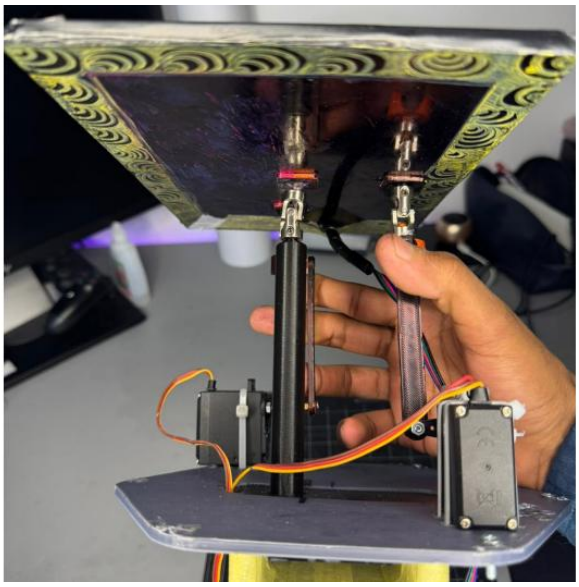

Photo of the RC Servo Motors and the mechanical linkage to the sensitive plate

Working Principle: Closed-Loop Control

A standard RC servo is a self-contained, closed-loop positioning system [12]. It consists of a DC motor, a gear reduction train, a position sensor, and a control circuit. The Arduino's Servo library generates the required 50Hz PWM signal, where the pulse width (typically 1000-2000 μs) corresponds to the target angle.

In this project, the servos are initialized to a "flat" position (flatXAngle = 80, flatYAngle = 90) and adjusted based on the PID output:

```
// Servo Actuation Snippet
xServo.write(flatXAngle + xOutput);
yServo.write(flatYAngle + yOutput);
```

5.2. L298N H-Bridge Driver (Mobile Base Actuation)

The mobile base uses three DC motors for propulsion, controlled by an **L298N Dual H-Bridge Motor Driver**.

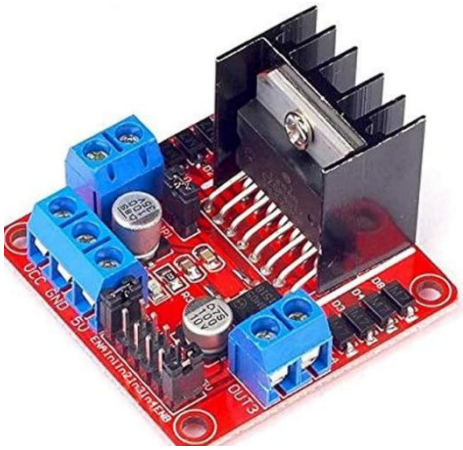

Photo of the L298N Motor Driver module

Working Principle: H-Bridge Topology

The L298N contains two independent H-bridges, allowing it to control two DC motors bidirectionally. The Arduino firmware implements a setMotor function to abstract the direction and speed control:

```
// Motor Control Logic (C++)

void setMotor(int motor, int speed){
  if(speed > 0){
    digitalWrite(dir1Pins[motor], HIGH);
    digitalWrite(dir2Pins[motor], LOW);
    analogWrite(pwmPins[motor], speed);
  } else if(speed < 0){
    digitalWrite(dir1Pins[motor], LOW);
    digitalWrite(dir2Pins[motor], HIGH);
    analogWrite(pwmPins[motor], -speed);
  } else {
    analogWrite(pwmPins[motor], 0);
  }

}
```

The speed is controlled via PWM on the pwmPins, while the direction is determined by the complementary states of dir1Pins and dir2Pins.

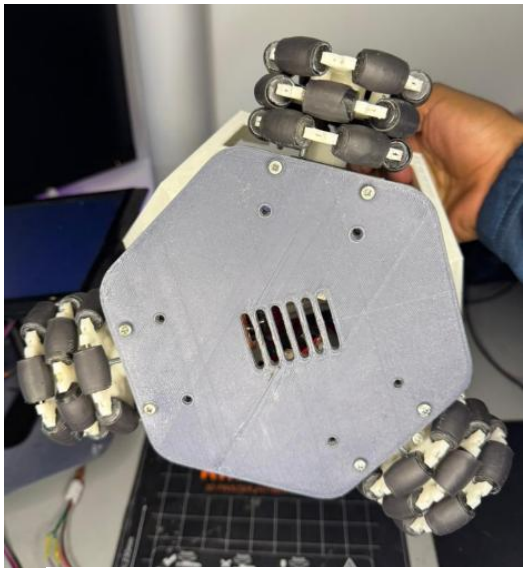

Photo of the three-wheeled omni-directional base chassis

6. Kinematics and Modeling

6.1. Omni-Directional Kinematics: The Inverse Model

The three-wheeled omni-directional robot is a **holonomic system**. The inverse kinematics model

translates the desired robot velocity $s = [\dot{x}, \dot{y}, \dot{\theta}]^T$ into the required angular velocity for each wheel, ω_i.

The provided implementation uses a specific wheel layout:

- **Motor 1**: Back
- **Motor 2**: Front Right
- **Motor 3**: Front Left

The inverse kinematics equations implemented in the ROS2 node are:

```
# Inverse Kinematics Implementation (Python)
```

```
# Motor 1: Back
self.w1 = ( vy + self.L * wz ) / self.r
# Motor 2: Front Right
self.w2 = ( (math.sqrt(3)/2) * vx - 0.5 * vy + self.L * wz ) / self.r
# Motor 3: Front Left
```

```
self.w3 = ( -(math.sqrt(3)/2) * vx - 0.5 * vy + self.L * wz ) / self.r
```

These equations account for the 120° geometry and the distance L from the center to each wheel.

6.2. Ball-on-Plate Modeling: Unstable Dynamics

The ball-on-plate system is modeled as a ball rolling without slipping. The linearized model shows that the ball's acceleration is proportional to the plate's tilt angle: $\ddot{x} \approx \frac{5}{7} g \phi_x$. This inherent instability requires a continuous, high-frequency control loop to maintain the ball at the setpoint.

7. Control System Design

7.1. Mobile Base Control: ROS2 and Serial Communication

The mobile base control is a **feed-forward system**. The ROS2 node calculates the wheel speeds and maps them to a 0-255 range for serial transmission:

```python
# Serial Transmission Logic
```

```python
def send_to_arduino(w1, w2, w3):
    def map_speed(s):
        # Map -10..10 rad/s -> 0..255 (128 is neutral)
        return int(max(min((s / 10.0) * 127 + 128, 255), 0))
```

```python
    ser.write(bytes([map_speed(w1), map_speed(w2), map_speed(w3)]))
```

The Arduino then decodes these bytes by subtracting 128 to recover the signed speed value.

7.2. Ball Balancing Control: PID Implementation

The ball balancing system uses a decoupled PID controller for the X and Y axes. The implementation includes integral windup protection (clipping) and derivative action to damp oscillations.

```cpp
// PID Calculation Logic
```

```cpp
errorX = setpointX - filteredX;
Px = Kpx * errorX;
Ix += Kix * errorX * dt;
Ix = clip2(Ix, -10, 10); // Integral Clipping
Dx = Kdx * (errorX - previousErrorX) / dt;
```

```cpp
float PIDx = Px + Ix + Dx;
```

The PID output is then mapped to the servo's angular range to physically adjust the plate.

8. Software Implementation

8.1. Raspberry Pi (ROS2 Node)

The ROS2 node OmniController handles the high-level logic, including:

- **Odometry Estimation**: Calculating the robot's position (x, y, θ) by integrating the wheel speeds over time.
- **TF Broadcasting**: Publishing the coordinate transform between the odom and base_link frames for visualization in RViz.

8.2. Arduino Firmware (Real-Time Control)

The Arduino firmware is split into two specialized sketches:

5 **Wheels Controlling**: Manages the L298N drivers and serial communication with the Raspberry Pi.
6 **Sensitive Plate**: Implements the high-frequency PID loop, touch panel sensing, and trajectory generation (e.g., circle, ellipse, four corners).

9. Experimental Results and Analysis

9.1. Sensor Performance Analysis

The resistive touch panel, combined with the moving average filter, provides a stable and high-resolution position signal. The sampling rate of 50Hz (20ms period) is sufficient for stabilizing the ball.

9.2. Actuator Performance Analysis

The RC servos provide the necessary torque to tilt the plate quickly. The L298N drivers effectively translate the PWM signals into motor power, though they are limited by their thermal capacity during prolonged operation.

9.3. Trajectory Tracking

The system demonstrates advanced capabilities by implementing trajectory tracking. By dynamically updating the setpointX and setpointY values, the ball can be made to follow complex paths such as circles or ellipses.

10. Discussion and Future Work

The developed platform successfully merges two distinct control challenges. The use of a resistive touch panel and RC servo motors emphasizes the principles of low-latency sensing and precise actuation.

Future work should focus on:

7 **Encoder Feedback**: Replacing the open-loop odometry estimation with actual encoder data for more accurate localization.
8 **IMU Integration**: Using an IMU to provide feed-forward compensation for the ball balancing system during base movement.

11. Conclusion

This report has presented a detailed scientific and technical analysis of an integrated robotic platform. The system's success relies on the careful selection and interfacing of its components: the **resistive touch panel** for high-speed position sensing, and the **RC servo motors** for precise angular actuation. The layered control architecture provides a robust and flexible framework for continued research in advanced robotics.

12. References

[1] Gregwar. Kinematics of omni-directional wheeled robots. *Gregwar's Blog*. https://gregwar.github.io/omnidirectional-wheeled-robots

[2] Ker, C. C., Lin, C. E., & Wang, R. T. (2007). Tracking and balance control of ball and plate system. *Journal of the Chinese Institute of Engineers*, 30(3), 459-470.

[3] Awtar, S., Bernard, C., Boklund, N., Master, A., Ueda, D., & Craig, K. (2002). Mechatronic design of a ball-on-plate balancing system.*Mechatronics*, 12(2), 217-228.

[4] Ali, E., & Aphiratsakun, N. (2015). AU Ball on Plate Balancing Robot.*Proceedings of the 2015 IEEE Conference on Robotics and Biomimetics*.

[5] Kopichev, M. M., Putov, A. V., & Pashenko, A. N. (2019). Ball on the plate balancing control system.*IOP Conference Series: Materials Science and Engineering*, 638(1), 012004.

[6] Ahmed, H., Zahid, S., Shahid, H., & Siddiq, F. (2024). Design and Control of Ball Balancing System.*9th MDSRIC*.

[7] Morales, L., Gordón, M., Camacho, O., Rosales, A., & Pozo, D. (2017). A comparative analysis among different controllers applied to the experimental ball and plate system.*2017 International Conference on Information Systems and Computer Science*.

[8] Development of a Ball and Plate System. (n.d.).*Research Material*.

[9] ROS 2 Control, Robot Control the Right Way. (n.d.).*Medium*. https://medium.com/@jiayi.hoffman/ros-2-control-robot-control-the-right-way-d0e72e7f1b6c

[10] How to Control a DC Motor With an L298 Controller and Raspberry Pi. (n.d.). *Maker Pro*. https://maker.pro/raspberry-pi/tutorial/how-to-control-a-dc-motor-with-an-l298-controller-and-raspberry-pi

[11] Riverdi. Resistive Touch Panel Construction and Working Principles. *Riverdi Blog*. https://riverdi.com/blog/resistive-touch-panel-construction-and-working-principles

[12] Circuit Digest. Servo Motor: Working Principle, Types & Control. *Circuit Digest*. https://circuitdigest.com/article/servo-motor-working-and-basics

Appendix A: Source Code

A.1. ROS2 Kinematics Node (Python)

```
import rclpy
```

```
from rclpy.node import Node
from geometry_msgs.msg import Twist
from nav_msgs.msg import Odometry
from tf2_ros import TransformBroadcaster
from geometry_msgs.msg import TransformStamped
import math
import time
import serial

# Serial to Arduino
ser = serial.Serial('/dev/ttyUSB0', 115200)

def send_to_arduino(w1, w2, w3):
    """Send wheel speeds to Arduino as bytes (0-255)"""
    def map_speed(s):
        # Map -10..10 rad/s -> 0..255
        return int(max(min((s / 10.0) * 127 + 128, 255), 0))
    ser.write(bytes([map_speed(w1), map_speed(w2), map_speed(w3)]))

class OmniController(Node):
    def __init__(self):
        super().__init__('omni_controller')
        self.r = 0.034   # Wheel radius (m)
        self.L = 0.09    # Distance from center to wheel
        self.x = 0.0
        self.y = 0.0
        self.theta = 0.0
        self.last_time = time.time()
        self.create_subscription(Twist, '/cmd_vel',
self.cmd_vel_callback, 10)
        self.odom_pub = self.create_publisher(Odometry, '/odom', 10)
        self.tf_broadcaster = TransformBroadcaster(self)
        self.timer = self.create_timer(0.02, self.update)
        self.w1 = 0.0
        self.w2 = 0.0
        self.w3 = 0.0

    def cmd_vel_callback(self, msg):
        vx = msg.linear.x
        vy = msg.linear.y
        wz = msg.angular.z
        self.w1 = ( vy + self.L * wz ) / self.r
```

```
        self.w2 = ( (math.sqrt(3)/2) * vx - 0.5 * vy + self.L * wz ) /
self.r
        self.w3 = ( -(math.sqrt(3)/2) * vx - 0.5 * vy + self.L * wz ) /
self.r
        send_to_arduino(self.w1, self.w2, self.w3)

    def update(self):
        current_time = time.time()
        dt = current_time - self.last_time
        self.last_time = current_time
        vx = (self.r / 3.0) * (2*self.w3 - self.w1 - self.w2)
        vy = (self.r / math.sqrt(3)) * (self.w1 - self.w2)
        wz = (self.r / (3*self.L)) * (self.w1 + self.w2 + self.w3)
        self.x += (vx * math.cos(self.theta) - vy * math.sin(self.theta))
* dt
        self.y += (vx * math.sin(self.theta) + vy * math.cos(self.theta))
* dt
        self.theta += wz * dt
        self.publish_odometry(vx, vy, wz)

    def publish_odometry(self, vx, vy, wz):
        odom = Odometry()
        odom.header.stamp = self.get_clock().now().to_msg()
        odom.header.frame_id = 'odom'
        odom.child_frame_id = 'base_link'
        odom.pose.pose.position.x = self.x
        odom.pose.pose.position.y = self.y
        qz = math.sin(self.theta / 2.0)
        qw = math.cos(self.theta / 2.0)
        odom.pose.pose.orientation.z = qz
        odom.pose.pose.orientation.w = qw
        odom.twist.twist.linear.x = vx
        odom.twist.twist.linear.y = vy
        odom.twist.twist.angular.z = wz
        self.odom_pub.publish(odom)
        t = TransformStamped()
        t.header.stamp = odom.header.stamp
        t.header.frame_id = 'odom'
        t.child_frame_id = 'base_link'
        t.transform.translation.x = self.x
        t.transform.translation.y = self.y
        t.transform.rotation.z = qz
        t.transform.rotation.w = qw
        self.tf_broadcaster.sendTransform(t)

def main(args=None):
    rclpy.init(args=args)
    node = OmniController()
    rclpy.spin(node)
```

```
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

A.2. Arduino: Wheels Controlling

```
#define WHEEL_COUNT 3

int pwmPins[WHEEL_COUNT] = {3, 6, 9};
int dir1Pins[WHEEL_COUNT] = {4, 7, 10};
int dir2Pins[WHEEL_COUNT] = {5, 8, 11};

void setup() {
  Serial.begin(115200);
  for(int i=0;i<WHEEL_COUNT;i++){
    pinMode(pwmPins[i], OUTPUT);
    pinMode(dir1Pins[i], OUTPUT);
    pinMode(dir2Pins[i], OUTPUT);
  }
}

void setMotor(int motor, int speed){
  if(speed > 0){
    digitalWrite(dir1Pins[motor], HIGH);
    digitalWrite(dir2Pins[motor], LOW);
    analogWrite(pwmPins[motor], speed);
  } else if(speed < 0){
    digitalWrite(dir1Pins[motor], LOW);
    digitalWrite(dir2Pins[motor], HIGH);
    analogWrite(pwmPins[motor], -speed);
  } else {
    analogWrite(pwmPins[motor], 0);
  }
}

void loop() {
  if(Serial.available() >= 3){
    int speeds[WHEEL_COUNT];
    for(int i=0;i<WHEEL_COUNT;i++){
      speeds[i] = Serial.read() - 128;
      setMotor(i, speeds[i]);
    }
  }
```

```
}
```

A.3. Arduino: Sensitive Plate

```
#include <stdint.h>
```

```
#include "TouchScreen.h"
#include <Servo.h>

#define YP A2
#define XM A3
#define YM 8
#define XP 9

float setpointX = 0, setpointY = 0;
const float width = 140, height = 250;
float time, timePrev, errorX, errorY, previousErrorX, previousErrorY;
TSPoint p;
float Px, Ix, Dx, Py, Iy, Dy;
int numValidPoints = 0, numInvalidPoints = 0;

const double Kpx = .5, Kix = 0.04, Kdx = .3;
const double Kpy = .4, Kiy = 0.05, Kdy = .2;

const int xServoPin = 10, yServoPin = 11;
Servo xServo, yServo;
const int flatXAngle = 80, flatYAngle = 90;

TouchScreen ts = TouchScreen(XP, YP, XM, YM, 500);
const int inputWindowSize = 10;
float readingsX[inputWindowSize], readingsY[inputWindowSize];
float sumX = 0, sumY = 0, filteredX = 0, filteredY = 0;

void setup() {
  Serial.begin(9600);
  xServo.attach(xServoPin);
  yServo.attach(yServoPin);
  xServo.write(flatXAngle);
  yServo.write(flatYAngle);
  time = millis();
}

void loop() {
  time = millis();
  float dt = (time - timePrev) / 1000;
  if (dt > 0.02) {
```

```
    timePrev = time;
    p = ts.getPoint();
    if(p.z == 0) { numInvalidPoints++; numValidPoints = 0; }
    else { numValidPoints++; numInvalidPoints = 0; }

    if(numInvalidPoints >= 100) {
      xServo.write(flatXAngle); yServo.write(flatYAngle);
      Ix = 0; Iy = 0; return;
    }
    if(numValidPoints < 3) return;

    if (p.z >= 10) {
      float x = map(p.x, 0, 1024, -82, 82);
      float y = map(p.y, 0, 1024, 75.5, -75.5);

      // Moving Average Filter
      sumX = sumX - readingsX[0];
      for(int i=0; i<inputWindowSize-1; i++) readingsX[i] =
readingsX[i+1];
      readingsX[inputWindowSize-1] = x; sumX += x; filteredX =
sumX/inputWindowSize;

      sumY = sumY - readingsY[0];
      for(int i=0; i<inputWindowSize-1; i++) readingsY[i] =
readingsY[i+1];
      readingsY[inputWindowSize-1] = y; sumY += y; filteredY =
sumY/inputWindowSize;

      errorX = setpointX - filteredX;
      errorY = setpointY - filteredY;

      Px = Kpx*errorX; Ix += Kix*errorX*dt; Dx = Kdx*(errorX-
previousErrorX)/dt;
      Py = Kpy*errorY; Iy += Kiy*errorY*dt; Dy = Kdy*(errorY-
previousErrorY)/dt;

      int xOutput = constrain(map(Px+Ix+Dx, -width/2, width/2, -50, 50),
-50, 50);
      int yOutput = constrain(map(Py+Iy+Dy, -height/2, height/2, -40,
40), -40, 40);

      xServo.write(flatXAngle + xOutput);
      yServo.write(flatYAngle + yOutput);
      previousErrorX = errorX; previousErrorY = errorY;
    }
  }

}
```